

DCL v0.5 Expansion Notes

This document captures proposed expansions beyond v0.1 for the Declarative Capability Language (DCL)

Focus areas:

- Policy enrichment (SLOs, quality attributes)
- Observability model
- Supervising lifecycles (orchestration/choreography)
- Expanded applicability (UI, queries, batch, file processing)
- Structural type system enhancements

1. Policy Expansion

Policies evolve to include:

- Reliability (retry, backoff)
- Performance (latency, throughput)
- Security (classification, sensitivity)
- Observability (logging, tracing, metrics)

Example:

```
policy ReliableProcessing {
  kind retry
  times 3
  delay 5s
  staggered delay 10s
  slo success rate at least 99.9%
}
```

2. Observability Model

Observability is declarative and references semantic constructs.

Example:

```
observe {
  rule TermsAccepted count violations as terms_not_accepted
  outcome Accepted count as registrations_completed
}
```

3. Supervising Lifecycles

Higher-level capabilities may own lifecycles spanning multiple capabilities.

Example:

```
capability JobManagement {

  lifecycle {
    begin step Pending
    step InProgress
    end step Completed
    end step Failed

    move Pending to InProgress on outcome JobStarted
    move InProgress to Completed on outcome JobCompleted
    move InProgress to Failed on outcome JobFailed
  }
}
```

4. Applicability Extensions

DCL can describe:

- UI behavior (state + actions)

- Queries (read-only capabilities)
- Batch jobs (scheduled actors)
- File processing pipelines

Example:

```
actor NightlyJob {
  kind scheduled_agent
}
```

5. Structural Types

New value types and modifiers:

- Id
- Token
- sensitive (modifier)

Example:

```
customerId: Id required
apiToken: Token sensitive required
```

6. Lifecycle Refinement

Streamlined syntax:

```
lifecycle {
  begin step Pending
  step InProgress
  end step Completed
  end step Failed
}
```